

Prompting and Complexity Modeling for Text-to-Cypher Generation

Nada Bou Kanaan
NLP Homework
Master in Computer Science

AI Assistance Disclaimer

Parts of this project have been developed with the assistance of OpenAI's Chat-GPT (GPT-5). The AI was used to support the development of project ideas, the structuring of methodological workflows, the drafting of descriptive texts, and the identification of relevant datasets and references. All content produced with AI assistance has been carefully reviewed, edited, and validated by me. I take full responsibility for the final content and its accuracy, relevance, and academic integrity.

1 Goal

Implement and evaluate different prompting strategies for Text-to-Cypher generation on the Neo4j **recommendations** graph, using Qwen as the base model. The project also includes a supervised Cypher difficulty classification baseline in order to strengthen the NLP component of the homework.

2 Problem

Text-to-Cypher systems must translate natural-language questions into executable graph queries while respecting schema constraints. In practice, this is difficult because small mistakes in labels, properties, filters, aggregation, or return fields can make a query incorrect even when it looks plausible.

Recent work highlights three useful directions. Chauhan et al., in *Mind the Query: A Benchmark Dataset towards Text2Cypher Task*, emphasize the need for realistic benchmark pairs of natural-language questions and validated Cypher queries. Hornsteiner et al., in *Real-Time Text-to-Cypher Query Generation with Large Language Models for Graph Databases*, stress the importance of strong schema grounding. Soleymani et al., in *Task Splitting and Prompt Engineering for Cypher Query Generation in Domain-Specific Knowledge Graphs*, propose decomposition as a way to make Cypher generation easier. Mandilara et al., in *Decoding the Mystery: How Can LLMs Turn Text Into Cypher in Complex Knowledge Graphs?*, focus on evaluation and schema-consistency issues.

The main question of this homework is therefore whether more complex prompting strategies actually improve Cypher generation compared with a simpler schema-aware baseline.

3 Data and Setup

The original benchmark contained 50 questions, but for runtime reasons the experiments were run on a reduced dataset of 30 queries. The subset preserves easy, medium, and hard questions, together with different query types such as retrieval, aggregation, verbose questions, and more complex graph patterns.

All experiments used the full schema of the Neo4j `recommendations` database. This was necessary because many benchmark questions require properties such as `runtime`, `budget`, `revenue`, `languages`, `imdbVotes`, `countries`, and `bio`.

The generation model used in the experiments was Qwen.

4 Prompting Strategies

4.1 Prompt 1: direct schema-aware prompting

In Prompt 1, the model receives the full schema, a few-shot set of examples, and the question, and then generates the Cypher query directly in one step.

This strategy is closest to the schema-aware setting discussed by Hornsteiner et al. It is also compatible with the benchmark philosophy of Chauhan et al., where the model is expected to generate correct Cypher under explicit schema constraints.

Strengths: simple, coherent, and fully grounded in the schema.

Weaknesses: may still omit important return fields or mis-handle more subtle semantic constraints.

4.2 Prompt 2: task splitting

In Prompt 2, the model first identifies the relevant schema elements and then generates the final Cypher query using those selected elements.

This strategy is directly inspired by Soleymani et al., who argue that decomposition can help align the question with the graph structure by breaking the generation into smaller subtasks.

Strengths: more controlled schema selection.

Weaknesses: mistakes in the first step can propagate into the final query.

4.3 Prompt 3: iterative refinement

In Prompt 3, the model first generates a Cypher query, then verifies it internally, and finally attempts to correct it.

This strategy is motivated by the validation-oriented perspective discussed by Mandilara et al., where Text-to-Cypher should be evaluated not only as generation, but also in terms of schema consistency and semantic correctness.

Strengths: allows self-correction.

Weaknesses: the same model acts as both generator and verifier, so deeper errors may not be repaired.

5 Evaluation

At first, exact match between generated and gold Cypher was considered as a metric. However, this is too strict, because two queries may differ in aliases or formatting while still being functionally equivalent.

For this reason, the final evaluation was execution-based. For each example:

1. the gold query was executed in Neo4j;
2. the predicted query was executed in Neo4j;
3. the prediction was counted as correct if it produced the same output as the gold query.

This criterion is more suitable for Text-to-Cypher because it measures functional correctness rather than textual similarity.

The notebook result files still store the generated Cypher strings together with parsing information and exact-match indicators. However, the final performance numbers reported in this homework are based on manual execution in Neo4j and output comparison with the gold queries, rather than on exact string matching alone.

6 Results

Using execution-based evaluation on the 30-query dataset, the results were:

- Prompt 1: 21/30 correct, equal to 70.0%
- Prompt 2: 16/30 correct, equal to 53.3%
- Prompt 3: 16/30 correct, equal to 53.3%

These results show that Prompt 1 was clearly the strongest of the three strategies. The direct schema-aware approach outperformed both task splitting and iterative refinement.

7 Discussion

The experiments show that increasing prompting complexity does not automatically improve performance. Prompt 1 worked best because it preserved the full semantic context of the question and schema in a single generation step. This is consistent with the emphasis on schema grounding found in Hornsteiner et al.

Prompt 2 did not improve over Prompt 1, even though it was inspired by the task-splitting strategy of Soleymani et al. In this homework setting, decomposition introduced an additional source of fragility, since an incomplete first step could negatively affect the final query.

Prompt 3 also failed to improve over Prompt 1. This suggests that self-correction alone is not sufficient when the same model is responsible for both initial generation and validation. In practice, the iterative-refinement setup did not reliably repair semantic mistakes.

8 Cypher Difficulty Classification Baseline

To complement the generation experiments, I also implemented a supervised neural baseline for Cypher difficulty classification. The task is to predict the structural level of a Cypher query, such as node-level, one-hop, two-hop, three-hop, or hard-level, from the query text alone.

The baseline was trained on the combined Cypher dataset. In this report, the combined dataset refers to the full collection obtained by merging the available Cypher queries from all difficulty levels into a single training and evaluation dataset. It includes node-level, one-hop, two-hop, three-hop, and hard-level queries, so the classifier learns from both simple and complex query structures.

The curated hard-level subset refers to a smaller manually selected subset containing only structurally difficult Cypher queries. These examples were separated from the broader combined dataset in order to test whether the classifier could still recognize the hard-level class when evaluated on more challenging queries rather than on the full mixed-difficulty distribution.

Each query was converted into a TF-IDF representation using unigrams and bigrams, and these vectors were used as input to a multilayer perceptron with two hidden layers, ReLU activations, and dropout. The network was trained with cross-entropy loss.

Results:

- On the combined dataset: Accuracy = 0.9727, F1 = 0.9529
- On the curated hard-level subset: Accuracy = 0.972, Precision = 1.0000

This experiment shows that Cypher difficulty can be predicted very effectively even with a relatively simple from-scratch neural architecture. It therefore complements the main generation task by showing that Cypher queries contain strong textual signals of structural complexity.

9 Proposed Prompt 4

After evaluating the three prompts, I proposed a new hybrid Prompt 4. Its design was based on the strengths and weaknesses of the previous strategies.

From Prompt 1, it inherits strong schema grounding. From Prompt 2, it inherits the idea that the model should internally identify the minimal relevant schema elements before writing the query. From Prompt 3, it inherits the idea of internal verification before producing the final answer.

At the same time, Prompt 4 avoids the main weaknesses of Prompt 2 and Prompt 3: it does not expose explicit intermediate outputs, and it does not rely on a long multi-step correction process. Instead, it keeps planning and verification internal and produces only one final Cypher query.

Prompt 4 was evaluated on the 30-query dataset. It produced 4 incorrect outputs and therefore correctly solved 26 out of 30 queries, achieving 86.7% accuracy. Compared with the previous prompting strategies, this is the best result obtained in the project and suggests that the hybrid design successfully combines the strongest aspects of the earlier prompts.

10 Conclusion

This homework shows that execution-based evaluation is more appropriate than exact string matching for Text-to-Cypher generation. Among the three initially tested prompting strategies, direct schema-aware prompting achieved the best result, while task splitting and iterative refinement did not improve performance for Qwen on this dataset.

The additional Cypher difficulty classification experiment showed that the structural complexity of Cypher queries can be modeled effectively by a supervised neural baseline trained from scratch. Finally, based on the strengths and weaknesses of the three prompting strategies, a fourth hybrid prompt was designed and tested on the hardest residual cases, where it produced encouraging results.